Tutorial 1

Open `src/tutorial-1/Tutorial1.tsx` to get started.

How Interactive Formulas are Implemented

What are interactive formulas?

An interactive formula is one that lets readers play around with the computations it implies or walks through an explanation of that formula step-by-step. They are intended to give readers new kinds of opportunities to look into the meaning of a formula.

What this tutorial covers

This tutorial walks you through how to build interactive formulas using a custom library designed for this purpose. By the end, you'll be able to take a LaTeX equation, label its variables, wire up the math, and let readers drag values to see what happens.



A note on feedback

As you work through this tutorial, **please think aloud** — narrate what you're doing, what you expect to happen, what you find confusing and what you notice along the way. There are no wrong observations. We're interested in hearing your thought process as you learn the library, not just the end result.

Step 1: Define a formula config

In this library, everything you specify about how to make a formula interactive goes into a "Config" object. **Create one such object now.** It contains three parts — `formulas`, `variables`, and `semantics` — which you will fill out in the following steps:

```
const config: Config = {
  formulas: [...],
  variables: {...},
  semantics: function ({ vars }) {...},
};
```

In brief, these sections will contain:

- `formulas` — the LaTeX equation(s) to display
- `variables` — specifications for annotations and interactions for variables
- `semantics` — a function that defines how compute some variables's values from others'

Step 2: Write the formula

First, draw a formula on the page. Formulas are written in LaTeX. **Add the following entry to the `formulas` array:**

```
formulas: [
  {
    id: "gravity",
    latex: "\\vec{F} = G \\frac{m_1 m_2}{r^2}",
  },
],
```

Write the formula in LaTeX as an entry in the `formulas` array. Assign the entry a unique `id`. You'll need to refer to this `id` later.

Step 3: Display the formula

Render the formula on screen so you can see your progress as you build. The library provides two React components for this: `Provider` (which makes the config available to

everything inside it) and `Formula` (which renders a specific equation). **Replace your app's return statement with the following JSX:**

```
<Provider config={config}>
  <Formula id="gravity" />
</Provider>
```

As seen above, you must wrap the `Formula` component in a `Provider` that passes the config to the formulas. Make sure the `id` for the Formula matches the ID you specified in the `formulas` entry in the config.

Then, `npm run dev` in your terminal to start the development server. Then, open the URL it prints. You should see the equation rendered on screen — but the formula is just static now. You'll change that shortly.
Keep the dev server running; it will update automatically as you make changes in the following steps.

Step 4: Label your variables

Next, start to explain the formula by labeling the variables with descriptive names and example values. Let's start by labeling a single variable.

4a: Give the variable a `name`

Start by giving a name to just one variable — `m_2`; in our running example, this will represent the mass of a person. To do this, **add a `variables` object with a single entry:**

```
variables: {
  m_2: {
    name: "Mass of Person",
  },
},
```

Let's pick this apart. `variables` is a dictionary, which contains keys for variables and properties for how they will be labeled and made interactive. The above dictionary has one key, `m_2`. This is should be the exact LaTeX of the variable you want to annotate as it appears in the formula (case-sensitive).

`name` is the plaintext name that will be shown next to the variable in the rendered formula.

Reload your page to see `m_2` labeled with "Mass of Person."

This step demonstrates something additionally important: you don't have to define all variables in a formula, but rather just those you want to explain and/or make interactive.

4b: Provide an example value

Show an example value for the variable by providing a `default` property:

```
variables: {
  m_2: {
    default: 80,
    name: "Mass of Person",
  },
},
```

Reload the page to see the default value showing in the label. Here, the value is meant to suggest that in an upcoming example computation, the mass of a person is assumed to be 80 kg.

4c: Label more variables

Now, add a variable representing the distance between the two masses `r`. In the worked example in the formula, this is meant to represent Earth's radius. Your adjusted `variables` object should look like this:

```
variables: {
  m_2: {
    default: 80,
    name: "Mass of Person",
  },
  r: {
    default: 6.371e6,
    name: "Earth's radius",
```

```
  },
},
```

4d: Label even more variables

For an interactive formula, you don't need to annotate all the variables. But in this example, it makes sense to do so, so go ahead and do it. Update the variables listing to look like this:

```
variables: {
  "\\vec{F}": {
    name: "Gravitational Force",
  },
  G: {
    default: 6.674e-11,
    name: "Gravitational Constant",
  },
  m_1: {
    default: 5.972e24,
    name: "Mass of Earth",
  },
  m_2: {
    default: 80,
    name: "Mass of Person",
  },
  r: {
    default: 6.371e6,
    name: "Earth's radius",
  },
},
```

A couple of important notes here:

- When a variable is required complex LaTeX — like space or macros (e.g., `\vec`), you need to define the variable key in the `variables` object using quotes and double-

escaped backslashes. For instance, to annotate `\vec{F}`, the key must be written `"\\vec{F}"`

- As mentioned before names and default values are optional for all variables. The above example leaves out a default for `\\vec{F}` because, as you'll soon see, this value will be computed from the others

4e: Format example values

Configure how example values are shown to increase legibility of those values. The library provides two options for doing so.

Significant figures: You can limit the number of significant figures (i.e., the number of digits displayed starting from the first non-zero digit) using `sigFigs`. Try to do so for `G` and `m_1` like so:

```
G: {
  default: 6.674e-11,
  name: "Gravitational Constant",
  sigFigs: 3,
},
m_1: {
  default: 5.972e24,
  name: "Mass of Earth",
  sigFigs: 3,
},
```

For this example, `sigFigs: 3` results in the value of `G` dispaying as `6.67 × 10⁻¹¹` (3 significant figures: "6", "6", "7") rather than showing all available digits.

Precision: Another option is to define how many digits should be shown after the decimal place with `precision`, like so:

```
m_2: {
  default: 80,
  name: "Mass of Person",
  precision: 0,
},
```

Here, `precision: 0` shows `m_2` as a whole number, i.e., `80` with no decimal places. These two options conflict. If you define both sigFigs and precision on the same variable, sigFigs takes precedence.

Full variables config: Apply formatting to all example values like so, then refresh the page and make sure that the values appear right in the rendered formula:

```
variables: {
  "\\vec{F}": {
    name: "Gravitational Force",
  },
  G: {
    default: 6.674e-11,
    name: "Gravitational Constant",
    sigFigs: 3,
  },
  m_1: {
    default: 5.972e24,
    name: "Mass of Earth",
    sigFigs: 3,
  },
  m_2: {
    default: 80,
    name: "Mass of Person",
    precision: 0,
  },
  r: {
    default: 6.371e6,
    name: "Earth's radius",
    sigFigs: 3,
  },
},
```

Step 5: Define the computation behind the formula

To allow someone to play with a formula as an interactive calculator, there needs to be some definition of how input variable values map to output variable values. That mapping is defined with a `semantics` function. **Add the following `semantics` function to your config:**

```
semantics: function ({ vars }) {
  var product = vars.m_1 * vars.m_2;
  var squared = vars.r * vars.r;
  var force = (vars.G * product) / squared;
  vars["\\vec{F}"] = force;
},
```

Reading through this function: it gets the two masses and multiplies them, since gravitational force grows with how massive both objects are. It squares the distance, since force weakens with separation. It combines these into a fraction and scales by `G`, the universal gravitational constant. The last line stores the computed force back into `vars` so the formula can display it.

A semantics function takes as input a special `vars` object. This vars object is used to both read input variable values and set output variable values. Here, the input variables `G`, `m_1`, `m_2`, and `r` are read, in this case from the `default` values for the variables. The output variable `\\vec{F}` is set.

Variables are accessed and set using the same keys used in `variables`. You can use dot notation for simple and subscripted keys (`vars.G`, `vars.m_1`), though you will need to use bracket notation for variables that contain spaces and special characters (`vars["\\vec{F}"]`).

Step 6: Make a variable interactive

Let readers change a variable's value by dragging it, so they can explore how the formula responds. **Add `input: "drag"` to your `m_2` variable:**

```
m_2: {
  input: "drag",
  default: 80,
  name: "Mass of Person",
```

```
  precision: 0,
},
```

`input: "drag"` enables drag interaction on the variable — readers can now click and drag `m_2` to change its value. The `semantics` function does not need to change; it already reads from `vars.m_2`, so when the user drags the variable, the semantics function re-runs automatically and `\\vec{F}` updates in real time.

6a: Control the drag range and step size

A draggable variable needs to have specifications for bounds and increment. You can do this with properties `range` and `step`. **Add `range` and `step` to `m_2`:**

```
m_2: {
  input: "drag",
  default: 80,
  range: [1, 200],
  step: 1,
  name: "Mass of Person",
  precision: 0,
},
```

- `range: [1, 200]` — sets the minimum and maximum values the user can drag to
- `step: 1` — sets the increment size when dragging

6b: Make a second variable interactive

You can make multiple variables draggable. **Now do the same for `r`:**

```
r: {
  input: "drag",
  default: 6.371e6,
  range: [1e6, 1e7],
  step: 1e4,
  name: "Earth's radius",
```

```
  sigFigs: 3,
},
```

Here is the full `variables` config with `m_2` and `r` both interactive:

```
variables: {
  "\\vec{F}": {
    name: "Gravitational Force",
  },
  G: {
    default: 6.674e-11,
    name: "Gravitational Constant",
    sigFigs: 3,
  },
  m_1: {
    default: 5.972e24,
    name: "Mass of Earth",
    sigFigs: 3,
  },
  m_2: {
    input: "drag",
    default: 80,
    range: [1, 200],
    step: 1,
    name: "Mass of Person",
    precision: 0,
  },
  r: {
    input: "drag",
    default: 6.371e6,
    range: [1e6, 1e7],
    step: 1e4,
    name: "Earth's radius",
    sigFigs: 3,
```

```
  },
},
```

Helpfully, you do not need to change the semantics function as you change which variables are interactive. The semantics function reads all values, and when a variable is not interactive, `semantics` just reads a constant value off of the default.
Refresh the page and click and drag on `m_2` and `r`. You should see `F` update with an accurate computation. This `F` is continually computed using the `semantics` function whenever `m_2` or `r` is changed.

Reference

Config structure

The config has three main parts:

- `formulas` — the LaTeX equations to display
- `variables` — the variables and their properties
- `semantics` — a function that defines how variables relate to each other

Variable keys

The **variable key** is the single thread connecting the **LaTeX formula**, the **variables config**, and the **semantics function**.

Simple keys (like `r`, `G`) and subscripted keys (like `m_1`) use dot notation. Keys with LaTeX commands (like `\\vec{F}`) must be quoted in the config and accessed with bracket notation in semantics.

How to read the table below: Each *column* is a type of variable key. Each *row* shows how that key appears in a different layer of the config. So if you want to know "how do I write a subscripted key in my semantics function?", find the `"m_1"` column and the **Semantics** row — the answer is `vars.m_1`.

Layer	Simple key `"r"`	Subscripted key `"m_1"`	LaTeX key `"\\vec{F}"`
LaTeX	`r` in `"...r^2"`	`m_1` in `"...m_1 m_2..."`	`\\vec{F}` in `"\\vec{F} = ..."`

Layer	Simple key `"r"`	Subscripted key `"m_1"`	LaTeX key `"\\vec{F}"`
Variables config	`r: { default: 6.371e6, ... }`	`m_1: { default: 5.972e24, ... }`	`"\\vec{F}": { name: "..." }`
Semantics	`vars.r`	`vars.m_1`	`vars["\\vec{F}"]`

Variable properties

Property	Description	Example	Required?
`name`	Explanatory label displayed to users	`name: "Mass of Earth"`	No
`default`	The starting value	`default: 5.972e24`	Required for variables not computed by semantics
`sigFigs`	Number of significant figures to display	`sigFigs: 3`	No
`precision`	Number of decimal places to display	`precision: 0`	No
`input`	How the user interacts with it	`input: "drag"`	For interactive variables
`range`	Min and max values for input variables	`range: [0, 100]`	For interactive variables
`step`	Increment size when dragging	`step: 0.1`	For interactive variables

Note: `sigFigs` and `precision` are mutually exclusive. If both are defined, `sigFigs` takes precedence.

Components

Component	Purpose
`Provider`	Wraps everything; receives the `config` prop.
`Formula`	Renders a formula by `id`; must match an entry in `formulas`.